CHAPTER VII

Apache Hive
A Petabyte Scale Data Warehouse Using Hadoop

1. INTRODUCTION TO HIVE
2. HIVEQL

1. Understand the Purpose and Architecture of Apache Hive — Why Hive was created, how it simplifies big data compared to writing raw MR programs and Pig Scripts - architecture of the Hive ecosystem.
2. Master the Hive Data Model and Language (HiveQL) — Familiar with Hive's data structures and learn how to manipulate them through HiveQL.
3. Develop Practical Skills in Writing and Running Hive Queries — Learn to write, execute, and troubleshoot Hive queries.

Section N° 1

Introduction to Hive

- Pcqrpgarcb npmep_kkgle kmbcj
- Only two phases
- Job chain for long data flow
- Too many lines of code even for simple logic

- Not SQL-Based
  Uses Pig Latin, which is procedural and unfamiliar to traditional SQL users.
- Limited BI Tool Support
  Not directly compatible with reporting tools that use SQL/JDBC.
- Harder for Structured Querying
  Less convenient for performing joins, groupings, and aggregations on structured data.

?n_afc Fgtc gq _ b_r_ u_pcfmsqc qwqrck `sgjr ml rmn md F_bmmn rf_r _jjmuq sqcpq rm oscpw _lb _l_jwxc j_pec b_r_qcrq qrmpcb gl FBDQ sqgle _ QOJ+jgic j_les_ec a_jjcb FgtcOJ,

Introduction to Hive
What is Hive?

Section 01

Hive is a data warehouse infrastructure tool to process structured data in Hadoop

It resides on top of Hadoop to summarize Big Data, and makes querying and analyzing easy

Developed by Facebook, later the Apache Software Foundation took it up and developed it further as an open source under the name Apache Hive.

It is used by different companies. For example, Amazon uses it in Amazon Elastic MapReduce

It is not a relational database

It is not a design for OnLine Transaction Processing (OLTP)

It is not a language for real-time queries and row-level updates

It stores schema in a database and processed data into HDFS.

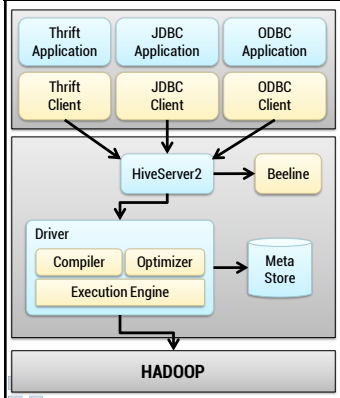
It is designed for OLAP.

It provides SQL type language for querying called HiveQL or HQL.

Connects to BI Tools: JDBC/ODBC support enables integration with data visualization and reporting platforms.

Introduction to Hive
Hive Architecture

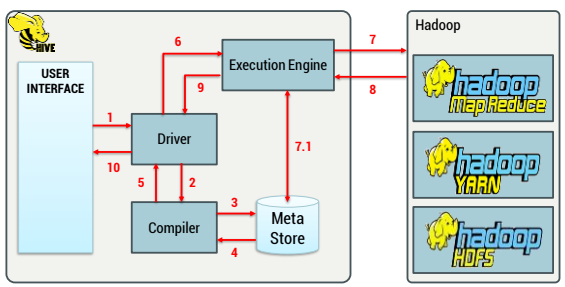
Section 01
6



Hive Client	Hive drivers support applications written in any language using JDBC, ODBC, and Thrift drivers, to perform queries on the Hive.
Hive Server 2	Provides clients with the ability to execute queries against the Hive
Meta Store	Hive uses relational database to store the schema or Metadata of tables, databases, columns in a table, their data types, and HDFS mapping.
Driver	The Hive driver receives the HiveQL statements submitted by the user through the command shell and creates session handles for the query.
Compiler	The execution plan generated by the compiler is based on the parse results. A DAG is created by the compiler where each step is a MR job, an operation on metadata, and a data manipulation step.
Optimizer	The optimizer splits the execution plan before performing the transformation operations so that efficiency and scalability are improved.
Execution Engine	The conjunction between Hive and MR. It processes the query and generates results as same as MR results. It uses the flavor of MR.

Introduction to Hive
Hive Query Processing

Section 01
7



1	Execute Query
2	Get Plan
3	Get Metadata
4	Send Metadata
5	Send Plan
6	Execute Plan
7	Execute Job
7.1	Metadata Ops
8	Fetch Result
9	Send Results
10	Send Results

Introduction to Hive
Hive Data Model

Section 01
8

DATABASE
- ✓ Analogous to database in relational DBs.
- ✓ Each database is a folder in HDFS.

TABLE
- ✓ Analogous to tables in relational DBs.
- ✓ Each table has corresponding directory in HDFS.

PARTITION
- ✓ Each Table can have one or more partition keys which determine how the data is stored.
- ✓ For example a table T with a date partition column ds had files with data for a particular date stored in the <table location>/ds=<date> directory in HDFS.
- ✓ Partitions allow the system to prune data to be inspected based on query predicates.

BUCKETS
- ✓ Data in each partition may in turn be divided into Buckets based on the hash of a given column.
- ✓ Each bucket is stored as a file in the partition directory.
- ✓ Bucketing allows the system to efficiently evaluate queries that depend on a sample of data (these are queries that use the SAMPLE clause on the table)

Hive Data Model - Example

```
CREATE TABLE salaries ( emp_id INT,  salary DOUBLE)
PARTITIONED BY (year INT, month INT)
CLUSTERED BY (emp_id) INTO 4 BUCKETS
STORED AS ORC;
```

```
salaries/
├── year=2024/
│   ├── month=1/
│   │   ├── bucket_00000
│   │   ├── bucket_00001
│   │   ├── bucket_00002
│   │   └── bucket_00003
│   └── month=2/
│       ├── bucket_00000
│       ├── bucket_00001
│       ├── bucket_00002
│       └── bucket_00003
└── year=2025/
```

How to Choose the Number of Buckets

1. **Size of the Dataset :** Small dataset: 2–4 buckets. Medium dataset: 8–16 buckets. Large dataset: 32, 64, or more.
2. **Query Pattern:** If your queries frequently filter or join by a particular column, then bucketing on that column can improve performance.
3. **Parallelism:** The number of buckets determines the number of mappers/reducers that can process data in parallel. Align it with the number of mapper slots or cores in your cluster for optimal performance.
4. **Compatibility with Other Tables:** For bucket map joins, the number of buckets must be the same and bucketed on the same column in both tables.

Section N° 2

HiveQL

Definition & Philosophy

- HiveQL (Hive Query Language) is a SQL-like language used to interact with data stored in Hadoop through Apache Hive.
- It allows users to write queries similar to SQL, which Hive translates into execution plans using MapReduce, Tez, or Spark.

- **Declarative, not procedural** → You specify what you want, not how to compute it.
- **Bridges SQL and Big Data** → Designed for analysts and data engineers who know SQL but are unfamiliar with Java or MapReduce.

Design Principles

Abstraction over complexity
HiveQL hides the complexity of distributed computation by automatically compiling queries into execution plans.

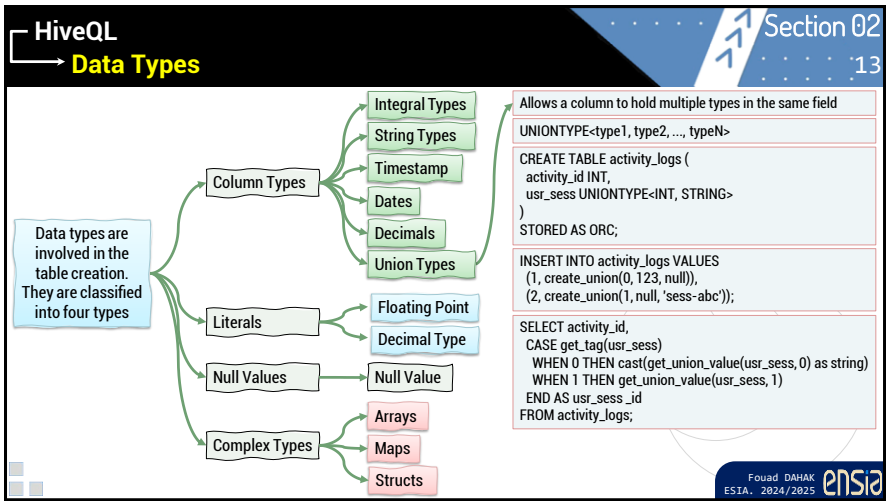
Extensibility
UDFs (User Defined Functions) and SerDes (Serializers/Deserializers) allow custom logic and data formats.

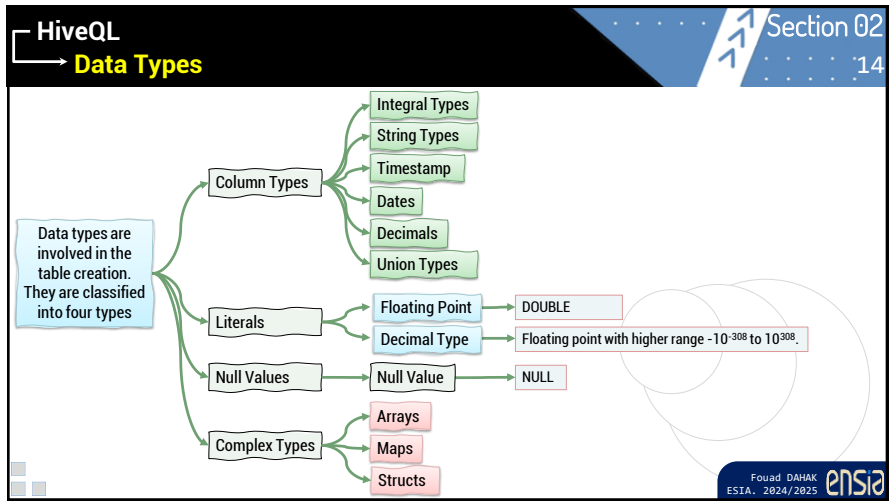
Integration with Hadoop stack
Hive works seamlessly with HDFS, YARN, Tez, and various file formats.

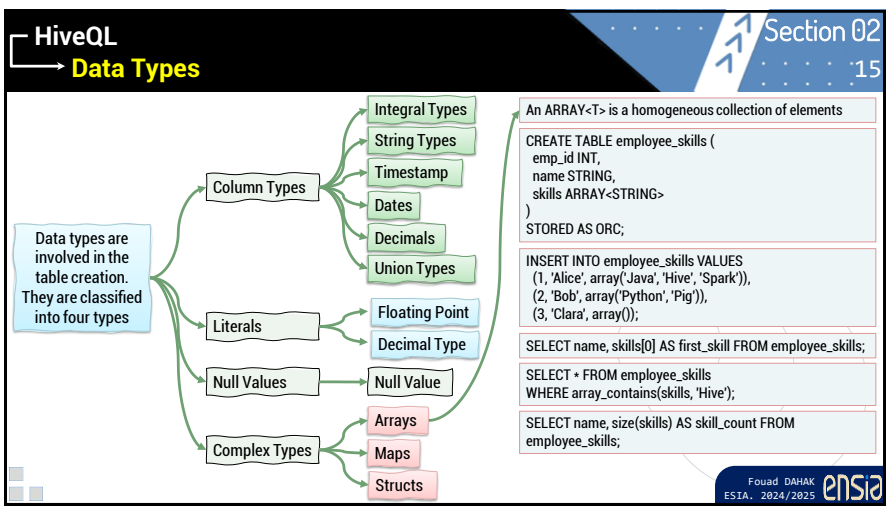
Data Types

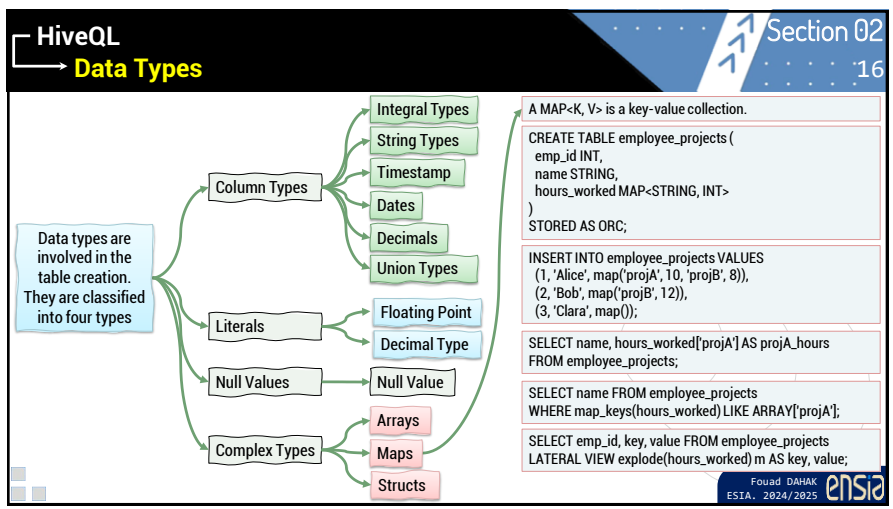
Data types are involved in the table creation. They are classified into four types

- Column Types
  - Integral Types → INT, BIGINT, SMALLINT, TINYINT
  - String Types → CHAR, VARCHAR
  - Timestamp
  - Dates
  - Decimals → DECIMAL(precision, scale)
  - Union Types
- Literals
  - Floating Point
  - Decimal Type
- Null Values
  - Null Value
- Complex Types
  - Arrays
  - Maps
  - Structs

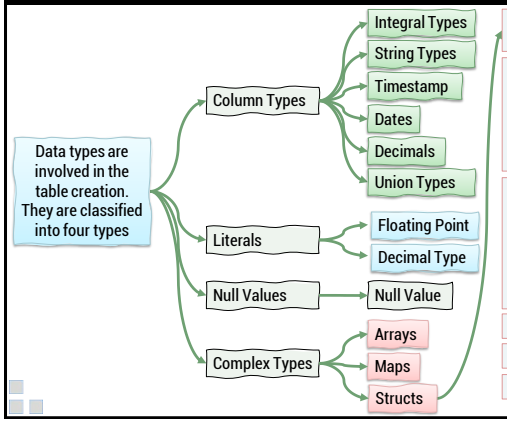

HiveQL
Data Types
Section 02
13
Data types are involved in the table creation. They are classified into four types
Column Types
Integral Types
String Types
Timestamp
Dates
Decimals
Union Types
Literals
Floating Point
Decimal Type
Null Values
Null Value
Complex Types
Arrays
Maps
Structs
Allows a column to hold multiple types in the same field
UNIONTYPE<type1, type2, ..., typeN>
CREATE TABLE activity_logs (
activity_id INT,
usr_sess UNIONTYPE<INT, STRING>
)
STORED AS ORC;
INSERT INTO activity_logs VALUES
(1, create_union(0, 123, null)),
(2, create_union(1, null, 'sess-abc'));
SELECT activity_id,
CASE get_tag(usr_sess)
WHEN 0 THEN cast(get_union_value(usr_sess, 0) as string)
WHEN 1 THEN get_union_value(usr_sess, 1)
END AS usr_sess _id
FROM activity_logs;
Fouad DAHAK
ESIA. 2024/2025
ensia


HiveQL
Data Types
Section 02
14
Data types are involved in the table creation. They are classified into four types
Column Types
Integral Types
String Types
Timestamp
Dates
Decimals
Union Types
Literals
Floating Point
DOUBLE
Decimal Type
Floating point with higher range -10-308 to 10308.
Null Values
Null Value
NULL
Complex Types
Arrays
Maps
Structs
Fouad DAHAK
ESIA. 2024/2025
ensia


HiveQL
Data Types
Section 02
15
Data types are involved in the table creation. They are classified into four types
Column Types
Integral Types
String Types
Timestamp
Dates
Decimals
Union Types
Literals
Floating Point
Decimal Type
Null Values
Null Value
Complex Types
Arrays
Maps
Structs
An ARRAY<T> is a homogeneous collection of elements
CREATE TABLE employee_skills (
emp_id INT,
name STRING,
skills ARRAY<STRING>
)
STORED AS ORC;
INSERT INTO employee_skills VALUES
(1, 'Alice', array('Java', 'Hive', 'Spark')),
(2, 'Bob', array('Python', 'Pig')),
(3, 'Clara', array());
SELECT name, skills[0] AS first_skill FROM employee_skills;
SELECT * FROM employee_skills
WHERE array_contains(skills, 'Hive');
SELECT name, size(skills) AS skill_count FROM
employee_skills;
Fouad DAHAK
ESIA. 2024/2025
ensia


HiveQL
Data Types
Section 02
16
Data types are involved in the table creation. They are classified into four types
Column Types
Integral Types
String Types
Timestamp
Dates
Decimals
Union Types
Literals
Floating Point
Decimal Type
Null Values
Null Value
Complex Types
Arrays
Maps
Structs
A MAP<K, V> is a key-value collection.
CREATE TABLE employee_projects (
emp_id INT,
name STRING,
hours_worked MAP<STRING, INT>
)
STORED AS ORC;
INSERT INTO employee_projects VALUES
(1, 'Alice', map('projA', 10, 'projB', 8)),
(2, 'Bob', map('projB', 12)),
(3, 'Clara', map());
SELECT name, hours_worked['projA'] AS projA_hours
FROM employee_projects;
SELECT name FROM employee_projects
WHERE map_keys(hours_worked) LIKE ARRAY['projA'];
SELECT emp_id, key, value FROM employee_projects
LATERAL VIEW explode(hours_worked) m AS key, value;
Fouad DAHAK
ESIA. 2024/2025
ensia

HiveQL → Data Types



A STRUCT<field1: T1, field2: T2, ...> is a named, nested group of fields, each with its own data type and label.

```
CREATE TABLE emp_contact (
  emp_id INT,
  name STRING,
  address STRUCT<street:STRING, city:STRING, zip:STRING>
)
STORED AS ORC;
```

```
INSERT INTO emp_contact VALUES
  (1, 'Alice', named_struct('street', '123 Main St', 'city', 'Algiers', 'zip', '16000')),
  (2, 'Bob', named_struct('street', '45 Rue Didouche', 'city', 'Oran', 'zip', '31000')),
  (3, 'Clara', named_struct('street', null, 'city', 'Annaba', 'zip', '23000'));
```

```
SELECT emp_id, name, address.city AS city FROM emp_contact;
```

```
SELECT emp_id, address FROM emp_contact;
```

```
SELECT address.street, address.zip FROM emp_contact;
```


HiveQL → Data Types - Example


```
CREATE TABLE employee (
  emp_id INT,
  name STRING,
  skills ARRAY<STRING>,
  project_hours
MAP<STRING,INT>,
  address STRUCT
<street:STRING, city:STRING,
zip:STRING>
)
STORED AS ORC;
```

```
INSERT INTO
employee_profile VALUES
(1,'Alice',
array('Java', 'Hive',
'Spark'),
map('projA', 10, 'projB',
8),
named_struct('street','123
Main St','city','Algiers',
'zip','16000')
);
```

```
INSERT INTO
employee_profile VALUES
(2,'Bob',
array('Python','Pig'),
map('projB',12),
named_struct('street','45
Rue
Didouche','city','Oran',
'zip','31000')
);
```

```
INSERT INTO
employee_profile VALUES
(3,'Clara',
array(),
map(),
named_struct('street',null
,'city','Annaba','zip','23
000')
);
```

```
SELECT * FROM employee
```

emp_id	name	skills	project_hours	address
1	Alice	["Java", "Hive", "Spark"]	{"projA":10,"projB":8}	{"street":"123 Main St", "city":"Algiers", "zip":"16000"}
2	Bob	["Python", "Pig"]	{"projB":12}	{"street":"45 Rue Didouche", "city":"Oran", "zip":"31000"}
3	Clara	[]	{}	{"street":null, "city":"Annaba", "zip":"23000"}

HiveQL → Data Types - Example


```
CREATE TABLE employee (
  emp_id INT,
  name STRING,
  skills ARRAY<STRING>,
  project_hours
MAP<STRING,INT>,
  address STRUCT
<street:STRING, city:STRING,
zip:STRING>
)
STORED AS ORC;
```

```
INSERT INTO
employee_profile VALUES
(1,'Alice',
array('Java', 'Hive',
'Spark'),
map('projA', 10, 'projB',
8),
named_struct('street','123
Main St','city','Algiers',
'zip','16000')
);
```

```
INSERT INTO
employee_profile VALUES
(2,'Bob',
array('Python','Pig'),
map('projB',12),
named_struct('street','45
Rue
Didouche','city','Oran',
'zip','31000')
);
```

```
INSERT INTO
employee_profile VALUES
(3,'Clara',
array(),
map(),
named_struct('street',null
,'city','Annaba','zip','23
000')
);
```

```
SELECT emp_id, name, skills[0] AS first_skill FROM employee
```

emp_id	name	First_skills
1	Alice	Java
2	Bob	Python
3	Clara	NULL

HiveQL → Data Types - Example


```
CREATE TABLE employee (
  emp_id INT,
  name STRING,
  skills ARRAY<STRING>,
  project_hours
MAP<STRING,INT>,
  address STRUCT
<street:STRING, city:STRING,
zip:STRING>
)
STORED AS ORC;
```

```
INSERT INTO
employee_profile VALUES
(1,'Alice',
array('Java', 'Hive',
'Spark'),
map('projA', 10, 'projB',
8),
named_struct('street','123
Main St','city','Algiers',
'zip','16000')
);
```

```
INSERT INTO
employee_profile VALUES
(2,'Bob',
array('Python','Pig'),
map('projB',12),
named_struct('street','45
Rue
Didouche','city','Oran',
'zip','31000')
);
```

```
INSERT INTO
employee_profile VALUES
(3,'Clara',
array(),
map(),
named_struct('street',null
,'city','Annaba','zip','23
000')
);
```

```
SELECT emp_id, name, project_hours['projB'] AS hours_projB FROM employee
```

emp_id	name	hours_projB
1	Alice	8
2	Bob	9
3	Clara	NULL

HiveQL
Data Types - Example


```
CREATE TABLE employee (
  emp_id INT,
  name STRING,
  skills ARRAY<STRING>,
  project_hours
MAP<STRING,INT>,
  address STRUCT
<street:STRING, city:STRING,
zip:STRING>
)
STORED AS ORC;
```

```
INSERT INTO
employee_profile VALUES
(1,'Alice',
array('Java', 'Hive',
'Spark'),
map('projA', 10, 'projB',
8),
named_struct('street','123
Main St','city','Algiers',
'zip','16000')
);
```

```
INSERT INTO
employee_profile VALUES
(2,'Bob',
array('Python','Pig'),
map('projB',12),
named_struct('street','45
Rue
Didouche','city','Oran',
'zip','31000')
);
```

```
INSERT INTO
employee_profile VALUES
(3,'Clara',
array(),
map(),
named_struct('street',null
,'city','Annaba','zip','23
000')
);
```

SELECT emp_id, name, address.city AS city FROM employee

emp_id	name	city
1	Alice	Algiers
2	Bob	Oran
3	Clara	Annaba

HiveQL
Relational, Arithmetic And Logical Operators

OPERATOR	OPERAND TYPES	DESCRIPTION
A = B	all primitive types	TRUE if expression A is equivalent to expression B; otherwise FALSE
A != B	all primitive types	TRUE if expression A is not equivalent to expression B; otherwise FALSE
A < B	all primitive types	TRUE if expression A is less than expression B; otherwise FALSE
A <= B	all primitive types	TRUE if expression A is less than or equal to expression B; otherwise FALSE
A > B	all primitive types	TRUE if expression A is greater than expression B] otherwise FALSE
A >= B	all primitive types	TRUE if expression A is greater than or equal to expression B otherwise FALSE
A IS NULL	all types	TRUE if expression A evaluates to NULL otherwise FALSE
A IS NOT NULL	all types	FALSE if expression A evaluates to NULL otherwise TRUE
A LIKE B	strings	TRUE if string A matches the SQL simple regular expression B
A RLIKE B	strings	NULL if A or B is NULL, TRUE if any substring of A matches the Java regular expression B
A REGEXP B	strings	Same as RLIKE

A + B	all number types	Gives the result of adding A and B.
A - B	all number types	Gives the result of subtracting B from A.
A * B	all number types	Gives the result of multiplying A and B.
A / B	all number types	Gives the result of dividing B from A.
A % B	all number types	Gives the reminder resulting from dividing A by B.
A & B	all number types	Gives the result of bitwise AND of A and B.
A ^ B	all number types	Gives the result of bitwise XOR of A and B.
~A	all number types	Gives the result of bitwise NOT of A

A AND B	boolean	
A && B	boolean	Same as A AND B
A OR B	boolean	
NOT A	boolean	
!A	boolean	Same as NOT A

HiveQL
Operators on Complex Types

OPERATOR	OPERAND TYPES	DESCRIPTION
A[n]	A is an Array and n is an int	returns the nth element in the array A. The first element has index 0
M[key]	M is a Map<K, V> and key has type K	returns the value corresponding to the key in the map
S.x	S is a struct	returns the x field of S

Aggregation Functions

RETURN TYPE	AGGREGATION FUNCTION NAME (SIGNATURE)
BIGINT	count(*), count(expr), count(DISTINCT expr[, expr_.])
DOUBLE	sum(col), sum(DISTINCT col)
DOUBLE	avg(col), avg(DISTINCT col)
DOUBLE	min(col)
DOUBLE	max(col)

HiveQL
Built In Functions

RETURN TYPE	FUNCTION NAME (SIGNATURE)
BIGINT	round(double a)
BIGINT	floor(double a)
BIGINT	ceil(double a)
Double	rand(), rand(int seed)
String	concat(string A, string B,...)
String	substr(string A, int start)
String	substr(string A, int start, int length)
String	upper(string A)
String	ucase(string A)
String	lower(string A)
String	lcase(string A)
String	trim(string A)
String	ltrim(string A)

RETURN TYPE	FUNCTION NAME (SIGNATURE)
string	rtrim(string A)
string	regexp_replace(string A, string B, string C)
int	size(Map<K.V>)
int	size(Array)
value of	cast(as)
string	from_unixtime(int unixtime)
string	to_date(string timestamp)
int	year(string date)
int	month(string date)
int	day(string date)
string	get_json_object(string json_string, string path)

HiveQL
DDL – CREATE DATABASE

When you create a database in Hive — Hive creates a physical directory in HDFS where all related tables and save metadata

```
CREATE DATABASE IF NOT EXISTS company_db
    COMMENT 'Database for company HR data'
    LOCATION '/data/hive/company_db'
    WITH DBPROPERTIES (  'created.by'='admin',
                         'purpose'='hr'
                      );
```

1. Metadata is recorded in the Metastore
Hive stores database-level metadata (name, location, owner, etc.) in its Metastore.

2. A folder is created in HDFS
By default, Hive creates a folder under /user/hive/warehouse/:

HiveQL
DDL – CREATE TABLE


```
CREATE [EXTERNAL] TABLE [IF NOT EXISTS] table_name (
  column_name1 data_type [COMMENT 'column comment'],
  column_name2 data_type [COMMENT 'column comment'],
  ...
  complex_column ARRAY<element_type>,
  complex_column MAP<key_type, value_type>,
  complex_column STRUCT<field1:type1, field2:type2, ...>
)
[COMMENT 'table comment']
[PARTITIONED BY (col_name1 type1, col_name2 type2, ...)]
[CLUSTERED BY (col_name) INTO num_buckets BUCKETS]
[ROW FORMAT DELIMITED
  FIELDS TERMINATED BY 'char'
  COLLECTION ITEMS TERMINATED BY 'char'
  MAP KEYS TERMINATED BY 'char']
[STORED AS file_format]
[LOCATION 'hdfs_path']
[TBLPROPERTIES ('property_name'='property_value', ...)];
```

Clause	Description
EXTERNAL	Defines an external table (data is not deleted on DROP TABLE)
PARTITIONED BY	Defines partition columns
CLUSTERED BY ... INTO ...	Bucketing clause
ROW FORMAT DELIMITED ...	Specifies input/output formatting for text-based formats
STORED AS	File format (e.g., ORC, PARQUET, TEXTFILE)
LOCATION	Custom HDFS location
TBLPROPERTIES	Metadata key-value pairs

Schema-on-read: Hive doesn't validate data on load, it reads structure at query time.
Delimited text: Most common format (CSV).
Stored As: Defines file format (TEXTFILE, ORC, PARQUET, etc.)

HiveQL
DDL …

Statement
DROP DATABASE [IF EXISTS] db_name [RESTRICT|CASCADE]
ALTER DATABASE db_name SET DBPROPERTIES (...)
SHOW DATABASES [LIKE 'pattern']
DESCRIBE DATABASE [EXTENDED] db_name

Statement
DROP TABLE [IF EXISTS] table_name
TRUNCATE TABLE table_name
ALTER TABLE table_name RENAME TO new_name
SHOW TABLES [IN db_name] [LIKE 'pattern']
DESCRIBE [EXTENDED|FORMATTED] table_name

Statement
ALTER TABLE table_name ADD PARTITION (...)
ALTER TABLE table_name DROP PARTITION (...)
ALTER TABLE table_name PARTITION (...) RENAME TO (...)
SHOW PARTITIONS table_name
MSCK REPAIR TABLE table_name

Statement
ALTER TABLE table_name ADD COLUMNS (...)
ALTER TABLE table_name REPLACE COLUMNS (...)
ALTER TABLE table_name CHANGE col_old col_new TYPE
DESCRIBE table_name col_name

HiveQL
DML - Load Data from HDFS


```
LOAD DATA [LOCAL] INPATH 'path_to_file_or_dir'
[OVERWRITE]
INTO TABLE table_name
[PARTITION (partition_col = value, ...)];
```

Clause	Description
LOCAL	Loads from the **local file system** instead of HDFS
INPATH	Path to the file or directory (HDFS or local)
OVERWRITE	Replaces all existing data in the table or partition
INTO TABLE	Specifies the table to load into
PARTITION	If the table is partitioned, specify the partition to load data into

```
LOAD DATA INPATH '/shared/employees.csv'
INTO TABLE employees;
```

- ❑ File must already be in HDFS.
- ❑ Hive moves the file (not copies) for managed tables.
- ❑ For external tables, data stays where it is.

HiveQL → DML - SELECT Statement


```
SELECT [ALL | DISTINCT]
    select_expr [, select_expr ...]
FROM   table1
    [[[LEFT|RIGHT|FULL] OUTER]JOIN table2 ON condition
    [,...]
[WHERE condition]
[GROUP BY column1 [, column2 ...]]
[HAVING condition]
[ORDER BY column1 [ASC | DESC] [, column2 ...]]
[LIMIT number]
[LATERAL VIEW lateral_view_expression AS alias [, ...]]
```

Clause	Description
SELECT	Specifies the columns or expressions to return
ALL	Returns all rows, including duplicates)
DISTINCT	Returns only unique rows
FROM	Defines the table(s) to query
JOIN	INNER, LEFT OUTER, RIGHT OUTER
ON	Specifies the join condition
WHERE	Filters rows **before** grouping
GROUP BY	Aggregates rows by the specified column(s)
HAVING	Filters groups **after** aggregation
ORDER BY	Sorts the result (uses a single reducer by default)
LIMIT	Restricts the number of returned rows
LATERAL VIEW	Use with explode() or inline() to flatten arrays or maps

```
SELECT e.name, d.dept_name, COUNT(*) AS project_count
FROM employees e JOIN departments d ON e.dept_id = d.id
WHERE d.location = 'Algiers'
GROUP BY e.name, d.dept_name
HAVING COUNT(*) > 2
ORDER BY e.name ASC
```

```
SELECT emp_id, skill
FROM employee_profile
LATERAL VIEW explode(skills) s AS skill;
```

HiveQL → DML - …

Category	Statement	Description
Insert	INSERT INTO … / INSERT OVERWRITE	
Export/Import	EXPORT TABLE … / IMPORT TABLE …	Move tables between clusters using HDFS
Update	UPDATE table SET … WHERE …	Modify specific rows (supported with ACID tables)
Delete	DELETE FROM table WHERE …	Delete specific rows (only in ACID-compliant tables)
Merge	MERGE INTO …	Perform UPSERT operations based on match conditions)

```
MERGE INTO target_table AS t
USING source_table AS s
ON t.key = s.key
WHEN MATCHED THEN UPDATE SET t.col1 = s.col1, t.col2 = s.col2
WHEN NOT MATCHED THEN INSERT VALUES (s.col1, s.col2);
```


HiveQL → ACID-compliant table

To create an ACID-compliant table, you need to satisfy three key requirements:

1. Use a Transactional File Format
   Hive only supports ACID transactions on tables stored as ORC
2. Specify ACID Properties in TBLPROPERTIES
3. Enable Bucketing
   You must bucket the table on a primary key

```
CREATE TABLE employees (
    emp_id INT,
    name STRING,
    salary FLOAT
)
CLUSTERED BY (emp_id) INTO 4 BUCKETS
STORED AS ORC
TBLPROPERTIES ( 'transactional' = 'true' );
```


HiveQL → Views

A view in is a virtual table defined by a SELECT query.
- ❑ It does not store data itself.
- ❑ It reuses existing table(s) to present a filtered or transformed perspective of the data.

Simplify complex joins or filters

Provide logical abstraction for analysts

Restrict access to sensitive columns

```
SHOW VIEWS [IN database_name] [LIKE 'pattern'];
DESCRIBE FORMATTED view_name;
DROP VIEW IF EXISTS view_name;
```

```
CREATE [OR REPLACE] VIEW [IF NOT EXISTS]
view_name [(col1, col2, ...)]
AS
SELECT statement;
```

```
CREATE VIEW female_employees AS
SELECT emp_id, name, department
FROM employees
WHERE gender = 'FEMALE';
```

```
SELECT * FROM female_employees;
```

```
CREATE VIEW top_paid (employee_id, full_name,
salary_amount) AS
SELECT id, name, salary
FROM employees
WHERE salary > 100000;
```

Apache Hive

Apache Hive is a powerful data warehousing tool built on top of Hadoop, designed to enable SQL-like querying of large-scale datasets stored in HDFS. It translates HiveQL queries into MapReduce, Tez, or Spark jobs, abstracting the complexity of distributed processing from the user.

- **Ease of Use**: Hive makes Hadoop accessible to analysts and developers through a familiar SQL interface.
- **Scalability**: It handles petabytes of data efficiently using Hadoop's distributed storage and computation.
- **Schema-on-Read**: It does not require loading data into the system; Hive interprets data structure at query time.
- **Extensibility**: Supports custom UDFs, integration with Tez and Spark, and a variety of storage formats like ORC and Parquet.
- **Performance**: Features like partitioning, bucketing, vectorization, and cost-based optimization help improve performance for analytical workloads.

Not ideal for real-time queries or transactional workloads

Latency can be higher than traditional RDBMS systems due to the nature of batch processing.

Hive is best suited for batch processing of large volumes of data, especially for ETL, reporting, and data summarization tasks

Chapter :

Thank You

Q & A